

1. Финальные приготовления

Цели

- Полная готовность к работе с Git.

01 Установка имени и электронной почты

Если вы никогда ранее не использовали Git, для начала вам необходимо осуществить установку. Выполните следующие команды, чтобы Git узнал ваше имя и электронную почту. Эти данные используются для подписи изменений сделанных вами, что позволит отслеживать, кто и когда сделал изменения в файле.

Выполните

```
git config --global user.name "Your Name"
git config --global user.email "your_email@whatever.com"
```

02 Имя ветки по умолчанию

Мы будем использовать `main` в качестве имени ветки по умолчанию. Чтобы установить его, выполните следующую команду:

Выполните

```
git config --global init.defaultBranch main
```

03 Корректная обработка оконных строк

2. Создание проекта

Цели

- Научиться создавать Git-репозиторий с нуля.

01 Создайте страницу «Hello, World»

Начните работу в пустой директории (например, `repositories`, если вы скачали архив с предыдущего шага) с создания пустой поддиректории `work`, затем войдите в неё и создайте там файл `hello.html` с таким содержанием:

Выполните

```
mkdir work
cd work
touch hello.html
```

Файл: hello.html

```
Hello, World
```

02 Создайте репозиторий

Теперь у вас есть директория с одним файлом. Чтобы создать Git-репозиторий из этой директории, выполните команду `git init`.

3. Проверка состояния

Цели

- Научиться проверять состояние репозитория.

01 Проверьте состояние репозитория

Используйте команду `git status`, чтобы проверить текущее состояние репозитория.

Выполните

```
git status
```

Вы увидите:

Результат

```
$ git status
On branch main
nothing to commit, working tree clean
```

Если после выполнения предыдущей команды вы видите `On branch master` вместо `On branch main`, это означает, что у вас немного устаревшая версия Git'a, которая не поняла нас, когда мы попросили установить имя ветки по умолчанию на `main`. В этом случае вы можете переименовать ветку в `main` с помощью следующей команды:

4. Внесение изменений

Цели

- Научиться отслеживать состояние рабочей директории.

01 Измените страницу «Hello, World»

Добавим кое-какие HTML-теги к нашему приветствию. Измените содержимое файла на:

Файл: hello.html

```
<h1>Hello, World!</h1>
```

02 Проверьте состояние

Теперь проверьте состояние рабочей директории.

Выполните

```
git status
```

Вы увидите...

Результат

5. Индексация изменений

Цели

- Научиться индексировать изменения для последующих коммитов.

01 Добавьте изменения

Теперь дайте команду Git проиндексировать изменения. Проверьте состояние:

Выполните

```
git add hello.html
git status
```

Вы увидите:

Результат

```
$ git add hello.html
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   hello.html
```

Изменения файла `hello.html` были проиндексированы. Это означает, что Git теперь знает об изменении, но изменение пока не *перманентно* (читай, *навсегда*) записано в репозиторий. Следующий коммит будет включать в себя проиндексированные изменения.

6. Индексация и коммит

Отдельный шаг индексации в Git позволяет вам разделять большие изменения на маленькие коммиты. Аналогия: вы помыли машину и заодно залили жидкость для очистки стекла — эти два изменения по своей сути независимы, а потому лучше пометить их отдельно. В противном случае, в истории изменений бачка для жидкости очистки стекла будет запись "Помыл машину", что не соответствует сути изменения и может запутать того, кто потом будет разбираться в этой истории.

Предположим, что вы отредактировали три файла (`a.html`, `b.html`, и `c.html`). Теперь вы хотите закоммитить все изменения, при этом чтобы изменения в `a.html` и `b.html` были одним коммитом, в то время как изменения в `c.html` логически не связаны с первыми двумя файлами и должны идти отдельным коммитом.

В теории, вы можете сделать следующее:

```
git add a.html
git add b.html
git commit -m "Changes for a and b"
```

```
git add c.html
git commit -m "Unrelated change to c"
```

Разделяя индексацию и коммит, вы имеете возможность с легкостью настроить, что идет в какой коммит.

[← 5. Индексация изменений](#)[7. Коммит изменений →](#)

7. Коммит изменений

Цели

- Научиться коммитить изменения в репозиторий.

01 Закоммитьте изменения

Достаточно об индексации. Давайте сделаем коммит того, что мы проиндексировали, в репозиторий.

Когда вы ранее использовали `git commit` для коммита первоначальной версии файла `hello.html` в репозиторий, вы включили метку `-m`, которая делает комментарий в командной строке. Команда `commit` позволит вам интерактивно редактировать комментарии для коммита. Теперь давайте это проверим.

Если вы опустите метку `-m` из командной строки, Git перенесет вас в редактор по вашему выбору. Редактор выбирается из следующего списка (в порядке приоритета):

- переменная среды `GIT_EDITOR`
- параметр конфигурации `core.editor`
- переменная среды `VISUAL`
- переменная среды `EDITOR`

У меня переменная `EDITOR` установлена в `vim`. Если вы предпочитаете GUI-редактор, то теперь можно использовать VS Code в качестве Git-редактора.

8. Изменения, а не файлы

Цели

- Понять, что Git работает с изменениями, а не файлами.

Большинство систем контроля версий работает с файлами: вы добавляете файл в систему, и она отслеживает изменения файла с этого момента.

Git фокусируется на изменениях в файле, а не самом файле. Когда вы осуществляете команду `git add file`, вы не говорите Git добавить файл в репозиторий. Скорее вы говорите, что Git надо отметить текущее состояние файла, коммит которого будет произведен позже.

Мы попытаемся исследовать эту разницу в данном уроке.

01 Первое изменение: Добавьте стандартные теги страницы

Измените страницу «Hello, World», чтобы она содержала стандартные теги `<html>` и `<body>`.

Файл: hello.html

```
<html>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>
```


9. История

Цели

- Научиться просматривать историю проекта.

Получение списка произведенных изменений — функция команды `git log`.

Выполните

```
git log
```

Вы увидите:

Результат

```
$ git log
commit b7614c1aea1ffbc46400fe1a163842d6ec620a43
Author: Alexander Shvets <alex@github.com>
Date:   Tue Nov 28 05:51:38 2023 -0600

    Added HTML header

commit 46afaff2232fc3d564c40f65cb82e7e94839a1bb
Author: Alexander Shvets <alex@github.com>
Date:   Tue Nov 28 05:51:38 2023 -0600

    Added standard HTML page tags

commit 78433de967102f2b59d0a8a60eb397b2663ed282
```

10. Получение старых версий

Цели

- Научиться возвращать рабочую директорию к любому предыдущему состоянию.

Git позволяет очень просто путешествовать во времени, по крайней мере, для вашего проекта. Команда `checkout` обновит вашу рабочую директорию до любого предыдущего коммита.

01 Получите хеши предыдущих коммитов

Выполните

```
git log
```

Результат

```
$ git log
b7614c1 2023-11-28 | Added HTML header (HEAD -> main) [Alexander Shvets]
46afaff 2023-11-28 | Added standard HTML page tags [Alexander Shvets]
78433de 2023-11-28 | Added h1 tag [Alexander Shvets]
5836970 2023-11-28 | Initial commit [Alexander Shvets]
```

Просмотрите историю изменений и найдите хеш первого коммита. Он должен быть в последней строке результата `git log`. Используйте этот хеш (достаточно первых 7 символов) в команде ниже. Затем проверьте содержимое файла `hello.html`.

Выполните

11. Создание тегов версий

Цели

- Узнать, как создавать теги для коммитов для использования в будущем.

Думаю, вы согласитесь, что работать с хешами коммитов напрямую просто неудобно. Разве не было бы здорово, если бы вы могли обозначать конкретные коммиты понятными для человека названиями? Таким образом, вы могли бы четко видеть важные вехи в истории проекта. Кроме того, вы могли бы легко перейти к определенной версии проекта по ее названию. Именно для этого в Git придумали теги.

Давайте назовем текущую версию страницы `hello.html` первой, то есть `v1`.

01 Создайте тег первой версии

Выполните

```
git tag v1
git log
```

Результат

```
$ git tag v1
$ git log
b7614c1 2023-11-28 | Added HTML header (HEAD -> main, tag: v1) [Alexander Shvets]
46afaff 2023-11-28 | Added standard HTML page tags [Alexander Shvets]
78433de 2023-11-28 | Added h1 tag [Alexander Shvets]
5836970 2023-11-28 | Initial commit [Alexander Shvets]
```

12. Отмена локальных изменений (до индексации)

Цели

- Научиться отменять изменения в рабочей директории.

01 Переключитесь на ветку `main`

Убедитесь, что вы находитесь на последнем коммите ветки `main`, прежде чем продолжить работу.

Выполните

```
git switch main
```

02 Измените `hello.html`

Иногда после того как вы изменили файл в рабочей директории, вы передумали и хотите просто вернуться к тому, что уже было закоммичено. Команда `restore` справится с этой задачей.

Внесите изменение в файл `hello.html` в виде нежелательного комментария.

Файл: `hello.html`

```
<html>
  <head>
    <title>
```

13. Отмена проиндексированных изменений (перед коммитом)

Цели

- Научиться отменять изменения, которые были проиндексированы.

01 Измените файл и проиндексируйте изменения

Внесите изменение в файл `hello.html` в виде нежелательного комментария

Файл: `hello.html`

```
<html>
  <head>
    <!-- This is an unwanted but staged comment -->
  </head>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>
```

Проиндексируйте это изменение.

Выполните

```
git add hello.html
```